

Bien architecturer, bien dépenser

Well-Architected, optimisation des coûts et synthèse AWS

CL-AWS2 — Jour 6 (jour 33 du cursus) — dernier jour AWS avant Azure

Objectifs de la journée

À la fin de cette journée, vous serez capable de :

- Relire les **6 piliers Well-Architected** à la lumière de tout ce que vous avez construit depuis AWS1.
 - Expliquer le **modèle de tarification** des services clés (EC2, S3, RDS, Lambda, Fargate, réseau) et les **leviers d'économie** de chacun.
 - Utiliser les outils : **Cost Explorer, Budgets, tags, Trusted Advisor**.
 - **Auditer et réduire une facture réelle** — celle de Negoce+ (étude de cas chiffrée).
 - Comparer les 3 **architectures de référence** du cursus : 3-tiers, serverless, conteneurs.
 - Citer les **6R** de la migration vers le cloud.
 - Vous situer face au format d'examen **Cloud Practitioner** (grand quiz de synthèse, 20 questions).
-

Plan de la journée

1. **Well-Architected** — les 6 piliers, avec les exemples concrets du cursus.
2. **Comprendre sa facture** — modèle de tarification service par service, le piège du réseau.
3. **Les outils du contrôle des coûts** — Cost Explorer, Budgets, tags, Trusted Advisor.
4. **Étude de cas Negoce+** — une vraie facture à auditer et réduire (exercice fil rouge).
5. **Architectures de référence et migration** — 3-tiers vs serverless vs conteneurs ; les 6R.
6. **Grand quiz de synthèse AWS** — 20 questions format Cloud Practitioner, puis clôture du bloc.

⚠ Tous les tarifs de la journée sont **indicatifs** (eu-west-3, 2025) : les ordres de grandeur sont fiables, les centimes non. Réflexe professionnel : **AWS Pricing Calculator** avant tout engagement.

1. Well-Architected

Les 6 piliers — que vous pratiquez déjà sans le savoir

Le framework Well-Architected

Le **Well-Architected Framework** : la grille de lecture officielle d'AWS pour évaluer une architecture. Vu rapidement au J8 d'AWS1 — aujourd'hui, on le relit **avec votre vécu** :

- | | |
|------------------------------|--------------------------------|
| 1. Excellence opérationnelle | 4. Efficacité des performances |
| 2. Sécurité | 5. Optimisation des coûts |

3. Fiabilité

6. Durabilité

Pourquoi c'est important pour VOUS :

- C'est le squelette des examens **Cloud Practitioner et SAA** (bloc 17).
- C'est le langage des revues d'architecture en entreprise (« et côté pilier fiabilité ? »).
- L'outil **AWS Well-Architected Tool** (gratuit, en console) permet d'auditer une charge de travail avec cette grille.

Chaque pilier, une slide, deux exemples : un que vous avez **déjà fait**, un que vous **feriez en plus** en production.

Piliers 1 et 2 — Excellence opérationnelle, Sécurité

1. Excellence opérationnelle — *opérer par le code, apprendre de ses incidents.*

-  Déjà fait : scripts boto3 d'exploitation (TP2), user-data, CloudFormation (J5), teardown scriptés, logs centralisés CloudWatch.
-  En production : tout en pipeline (TP4), runbooks écrits, post-mortems sans blâme.

2. Sécurité — *moindre privilège, défense en profondeur, tout tracer.*

-  Déjà fait : IAM utilisateurs/groupes/MFA (AWS1 J1), SG en cascade (ALB → app → base au TP2), rôles d'exécution Lambda (J1), pas de secrets en dur.
-  En production : CloudTrail systématique, KMS, GuardDuty, revues d'accès — tout le bloc **CL-SECU** (5 jours) y est consacré.

Piliers 3 et 4 — Fiabilité, Efficacité des performances

3. Fiabilité — *survivre aux pannes : elles arriveront.*

-  Déjà fait : Multi-AZ partout au TP2 (sous-réseaux, ASG, RDS Multi-AZ), health checks ALB, ECS qui remplace une tâche morte (J4), rollback CloudFormation (J5).
-  En production : tests de panne réguliers, sauvegardes **restaurées** (pas juste faites), objectifs chiffrés (RTO/RPO).

4. Efficacité des performances — *le bon service, la bonne taille, mesurés.*

-  Déjà fait : choix t3.micro vs plus gros, tableau Lambda/Fargate/EC2 (J4), CloudFront pour le statique (AWS1 J7), DynamoDB pour le clé-valeur (J2).
-  En production : tests de charge, Compute Optimizer, réévaluer les choix quand AWS sort de nouveaux services (le « bon choix » périme).

Piliers 5 et 6 — Coûts, Durabilité

5. Optimisation des coûts — *payer pour ce qui sert, mesurer ce qu'on paie.*

-  Déjà fait : free-tier d'abord, encadrés 💰 dans chaque TP, teardowns, serverless payé à l'usage (J1-J3), calculs Fargate (J4).
-  En production : tags de facturation, plans d'engagement, revue mensuelle — **c'est la section 2 et l'étude de cas d'aujourd'hui.**

6. Durabilité — minimiser l'empreinte environnementale.

-  Déjà fait (sans le savoir) : éteindre ce qui ne sert pas, mutualiser (serverless, conteneurs = meilleure densité que des VM à 10 %).
-  En production : régions à énergie bas-carbone, dimensionnement au plus juste, graviton/ARM.

Remarquez : **coûts et durabilité se recouvrent presque** — le gaspillage coûte deux fois.

2. Comprendre sa facture

Le modèle de tarification, service par service

Les 3 familles de coûts — toujours les mêmes

Quelle que soit l'architecture, une facture AWS se décompose en trois familles :

Famille	On paie...	Exemples
Compute	du temps de calcul	EC2 (heure), Lambda (ms), Fargate (seconde)
Stockage	des Go conservés + des requêtes	S3, EBS, snapshots, RDS (stockage)
Réseau	des Go sortants et des heures de « tuyaux »	data transfer out, NAT GW, ALB

Les deux règles d'or :

1. **Le compute domine** la plupart des factures → c'est là qu'on optimise d'abord.
2. **Le réseau est le poste invisible** : personne ne le budgète, tout le monde le paie. On y revient.

EC2 : quatre façons de payer la même instance

Exemple : **m5.large** (2 vCPU, 8 Go), eu-west-3, tarifs indicatifs 2025 :

Modèle	Prix indicatif	Engagement	Cas d'usage
On-Demand	~0,112 \$/h (82 \$/mois)	aucun	pics, tests, courte durée
Savings Plans (1 an)	~30 % (~57 \$/mois)	1-3 ans, \$/h de compute	base stable — flexible (famille, taille, Fargate/Lambda inclus)
Reserved (1 an)	~35 % (~53 \$/mois)	1-3 ans, type précis	base stable, besoin figé

Modèle	Prix indicatif	Engagement	Cas d'usage
Spot	-60 à -90 % (~15-30 \$/mois)	aucun, mais interruptible (préavis 2 min)	batch, calcul reprennable, JAMAIS une base de données

La stratégie type en entreprise : **la charge de base en Savings Plans, les pics en On-Demand, le batch en Spot.**

Et le levier n°0, avant tous les autres : **éteindre** (une instance de dev allumée la nuit et le week-end = ~70 % de gâchis) et **redimensionner** (right-sizing).

S3 : des classes et un cycle de vie

Le stockage S3 se paie **au Go/mois + aux requêtes + aux récupérations** (tarifs indicatifs eu-west-3) :

Classe	\$/Go/mois	Particularité
Standard	~0,024	accès fréquent, aucun frais de lecture spécifique
Standard-IA	~0,013	accès occasionnel ; frais par Go récupéré, minimum 30 j
Glacier Instant Retrieval	~0,005	archive consultable instantanément ; min. 90 j
Glacier Deep Archive	~0,002	récupération en heures ; min. 180 j — l'archivage légal
Intelligent-Tiering	~0,024 puis auto	déplace tout seul selon l'usage (petits frais de suivi)

Le levier : les **règles de cycle de vie** (lifecycle) :

```
logs/ : Standard --30j--> Standard-IA --90j--> Glacier IR --365j--> suppression
```

100 Go de logs gardés 3 ans en Standard : ~86 *.Avec lifecycle* : * * 25^{**}. Même donnée, même durabilité (11 neufs), 3× moins cher.

Le piège : les données sortantes

La règle réseau d'AWS, à graver :

- Données **entrantes** : gratuites.
- Données **entre services de la même région** : gratuites ou presque (attention inter-AZ : ~0,01 \$/Go dans chaque sens).
- Données **sortantes vers Internet** : **~0,09 \$/Go** (eu-west-3, après 100 Go/mois gratuits).

Ça ne paraît rien ? Faites le calcul :

```
API qui sert 2 To/mois vers Internet : 2000 × 0,09 ≈ 180 $/mois
— souvent PLUS CHER que les serveurs qui la font tourner !
```

Les leviers : **CloudFront** (sortie CDN moins chère + cache = moins de Go), compresser les réponses, ne pas faire transiter les gros fichiers par l'API (liens S3 pré-signés, vus en J2).

Et son cousin : la **NAT Gateway** ($\sim 0,048 / h \approx * * 35 / \text{mois}^{**} + 0,048 \$ / \text{Go}$ traité) — indispensable en prod pour les sous-réseaux privés, hors de prix pour un lab. Vous l'avez éteinte au TP2, souvenez-vous.

Le modèle de chaque service, en une ligne

Récapitulatif des services du cursus — **connaître l'unité de facturation, c'est déjà savoir optimiser** :

Service	On paie	Levier principal
EC2	l'heure d'instance (+EBS)	éteindre, right-sizing, Savings Plans, Spot
EBS	le Go provisionné (même vide !)	supprimer volumes orphelins, gp2 → gp3 ($\sim -20\%$)
RDS	l'heure + stockage (+ $\times 2$ si Multi-AZ)	Multi-AZ en prod SEULEMENT, éteindre le dev, reserved
S3	Go stocké + requêtes + sortie	classes + lifecycle
Lambda	la requête + la ms$\times$Go	mémoire au plus juste, code rapide
Fargate	la seconde $\times$ vCPU/Go	tâches arrêtées, tailles au plus juste
DynamoDB	la requête (on-demand) ou capacité	on-demand si trafic irrégulier
ALB	l'heure ($\sim 19 \$ / \text{mois}$) + LCU	mutualiser, supprimer les inutilisés
NAT GW	l'heure ($\sim 35 \$ / \text{mois}$) + Go	1 par VPC suffit souvent ; endpoints VPC pour S3
CloudWatch	Go de logs ingérés + rétention	rétention limitée, filtrer ce qu'on loggue

3. Les outils du contrôle des coûts

Cost Explorer : voir où part l'argent

Cost Explorer (console → Billing and Cost Management) : l'outil d'analyse graphique de la facture.

- Vue par **service**, par **compte**, par **région**, par **tag** — jour par jour ou mois par mois.
- **Filtres et regroupements** : « montre-moi le coût EC2 du mois, groupé par type d'instance ».
- **Prévisions** : projection de fin de mois sur la tendance courante.
- Rapports intégrés : coûts mensuels par service, couverture Savings Plans, instances sous-utilisées.

Les 3 questions du réflexe mensuel d'un admin cloud :

1. Quels sont mes **5 premiers postes** de dépense ?
2. Qu'est-ce qui a **augmenté** depuis le mois dernier, et pourquoi ?
3. Qu'est-ce que je paie **qui ne sert à rien** (le samedi à 3 h du matin, ça consomme quoi ?) ?

Budgets et alertes : ne jamais découvrir la facture

Vu à AWS1 J8, jamais assez répété : **un budget avec alertes est le premier geste sur tout compte AWS.**

```
# Un budget mensuel de 50 $, alerte à 80 % (réel) et 100 % (prévu)
aws budgets create-budget --account-id 123456789012 \
  --budget file://budget.json \
  --notifications-with-subscribers file://notifications.json
```

- Alertes sur le coût **réel** ET sur le coût **prévisionnel** (forecast) — la seconde prévient AVANT le dépassement.
- **AWS Budgets Actions** peut même réagir tout seul (appliquer une politique IAM restrictive, arrêter des instances taguées).
- Complément : **CloudWatch billing alarms** et les **anomalies de coût** (Cost Anomaly Detection, détection automatique des dérives inhabituelles — gratuit, activez-le partout).

En formation, vos comptes sandbox ont un budget : c'est lui qui nous a signalé les NAT GW oubliées de la promo précédente. Histoire vraie.

Les tags : sans eux, une facture est illisible

Une facture de 10 000 \$ « EC2 » ne dit **rien**. La même facture ventilée par tags dit tout :

```
Projet=stockline   Env=prod   → 4 200 $   (OK, c'est le business)
Projet=stockline   Env=dev     → 2 800 $   (?! le dev coûte 2/3 de la prod ?)
(sans tag)         → 3 000 $   (personne ne sait ce que c'est)
```

La **stratégie de tags** minimale, à imposer dès le premier jour :

- **Projet**, **Environnement** (prod/recette/dev), **Propriétaire** (équipe ou trigramme), **CentreDeCout**.
- **Activés pour la facturation** (cost allocation tags, à cocher dans Billing) — sinon invisibles dans Cost Explorer.
- **Imposés techniquement** : tag policies / SCP au niveau de l'organisation — pas une règle orale.

Vous le vivez depuis AWS1 : votre **\$PREFIX** est un tag de propriétaire. Sans lui, impossible de savoir à qui est la ressource oubliée.

Trusted Advisor et Compute Optimizer

Deux services qui font l'audit à **votre place** :

Trusted Advisor — le contrôle technique du compte :

- Vérifications dans 5 domaines : **coûts**, performance, sécurité, tolérance aux pannes, quotas.
- Côté coûts : instances sous-utilisées, volumes EBS non attachés, IP élastiques inutilisées, load balancers sans trafic...

- Vérifications de base gratuites ; la liste complète nécessite un plan de support Business.

Compute Optimizer — le spécialiste du right-sizing (gratuit) :

- Analyse les métriques CloudWatch de vos instances/fonctions/volumes sur 14 jours et recommande : « cette m5.xlarge plafonne à 8 % de CPU → **m5.large**, économie ~50 % ».

Réflexe : ces outils **trouvent**, mais c'est vous qui **décidez et agissez** — d'où l'étude de cas qui arrive.



Démo 6-1 — Cost Explorer sur un vrai compte

Le formateur explore le compte de formation sous vos yeux.

1. Cost Explorer : coûts du mois par service — qui devine le premier poste ?
2. Zoom sur une journée de TP : la bosse EC2/RDS du TP2, le teardown visible le soir.
3. Filtrage par tag **Propriétaire** : la ressource oubliée est démasquée en 30 secondes.
4. Les mêmes chiffres en CLI (`aws ce get-cost-and-usage`) — parce que tout s'automatise.
5. Tour rapide : budget existant, alertes reçues, Trusted Advisor (volumes orphelins ?).

Fichier : `demos/08-cl-aws2/demo-6-1-cost-explorer.md`

Pendant la démo, notez : **quelles habitudes de la salle laissent des traces sur la facture ?** On en fait la liste ensemble juste après.

Les 10 gaspillages classiques (checklist d'audit)

À chercher **dans cet ordre** lors d'un audit de facture — du plus fréquent au plus sournois :

1. Instances **allumées 24/7** qui ne servent qu'aux heures ouvrées (dev, recette).
2. Instances **surdimensionnées** (CPU < 10 %) — right-sizing.
3. **Volumes EBS orphelins** (détachés mais provisionnés) et vieux **snapshots**.
4. **IP élastiques** non associées (payées quand elles ne servent pas !).
5. **NAT Gateways** multiples ou inutiles ; trafic S3 qui passe par la NAT (→ endpoint VPC gratuit).
6. **Load balancers** sans trafic (~19 \$/mois chacun).
7. S3 sans **lifecycle** : des To de logs en Standard depuis 3 ans.
8. **Multi-AZ et réplicas RDS sur le dev**.
9. Aucun **engagement** (Savings Plans) sur une base stable depuis 2 ans.
10. **gp2** au lieu de gp3, anciennes générations d'instances (m4 → m7 : moins cher et plus rapide).

Gardez cette liste : c'est votre grille pour l'étude de cas Negoce+.

4. Étude de cas — la facture de Negoce+

Exercice fil rouge : 3 architectures à auditer et optimiser

Negoce+ : le contexte

Vous connaissez **Negoce+** depuis le TP1 : PME de négoce (80 salariés), dont vous êtes l'équipe d'administration cloud. Trois charges de travail tournent sur AWS :

- **A. La plateforme StockLine de production** : 3-tiers (ALB + EC2 + RDS), migrée l'an dernier « vite fait » depuis le datacenter — dimensionnée large « au cas où ».
- **B. Les environnements de développement et recette** : copies quasi conformes de la prod, créées par l'ancien prestataire, jamais revues.
- **C. Le traitement nocturne** : consolidation des mouvements de stock + rapports, 2 h de calcul chaque nuit sur des instances dédiées.

La direction a reçu la facture d'octobre : **≈ 1 800 \$/mois**, en hausse constante. Le DAF vous convoque :

« Le cloud devait coûter moins cher que nos serveurs. Expliquez-moi cette facture, et revenez avec un plan pour la **réduire d'au moins 40 % sans dégrader la production**. »

La facture d'octobre (extrait fourni dans l'énoncé)

```
NEGOCE+ – Facture AWS octobre (extrait, eu-west-3, arrondis)
-----
A. PROD          2x m5.large on-demand 24/7 ..... 164 $
                  RDS db.m5.large Multi-AZ + 500 Go gp2 .... 352 $
                  ALB + 300 Go sortants + NAT GW (2) ..... 131 $
B. DEV/RECETTE   2x m5.large + 1x t3.large 24/7 ..... 232 $
                  RDS db.m5.large MULTI-AZ (dev !) ..... 301 $
                  8 volumes EBS dont 5 orphelins, 4 EIP .... 73 $
C. BATCH         2x c5.xlarge on-demand 24/7 (2 h/nuit !) . 296 $
DIVERS          S3 1,2 To tout en Standard, sans lifecycle 29 $
                  snapshots anciens, logs sans rétention ... 45 $
                  data transfer + inter-AZ divers ..... 180 $
-----
TOTAL ≈ 1 803 $/mois – détail complet et chiffres exacts dans l'énoncé
```

Rien d'exotique : **cette facture existe dans des centaines de PME**. Chaque ligne cache un des 10 gaspillages de la checklist.



Exercice 6-1 — Auditer et réduire (fil rouge, en binômes)

Binômes, 1 h 30 de travail + restitution. Le plat de résistance de la journée.

Pour chacune des 3 architectures (A, B, C) :

1. **Identifiez** les gaspillages (appuyez-vous sur la checklist des 10) ;
2. **Chiffrez** chaque économie proposée (calculs posés, tarifs fournis dans l'énoncé) ;
3. Classez vos mesures : **sans risque / à valider avec les équipes / structurelle** (changement d'architecture) ;
4. Produisez le **tableau final** : facture avant → après, % d'économie, en respectant la contrainte (« sans dégrader la prod »).

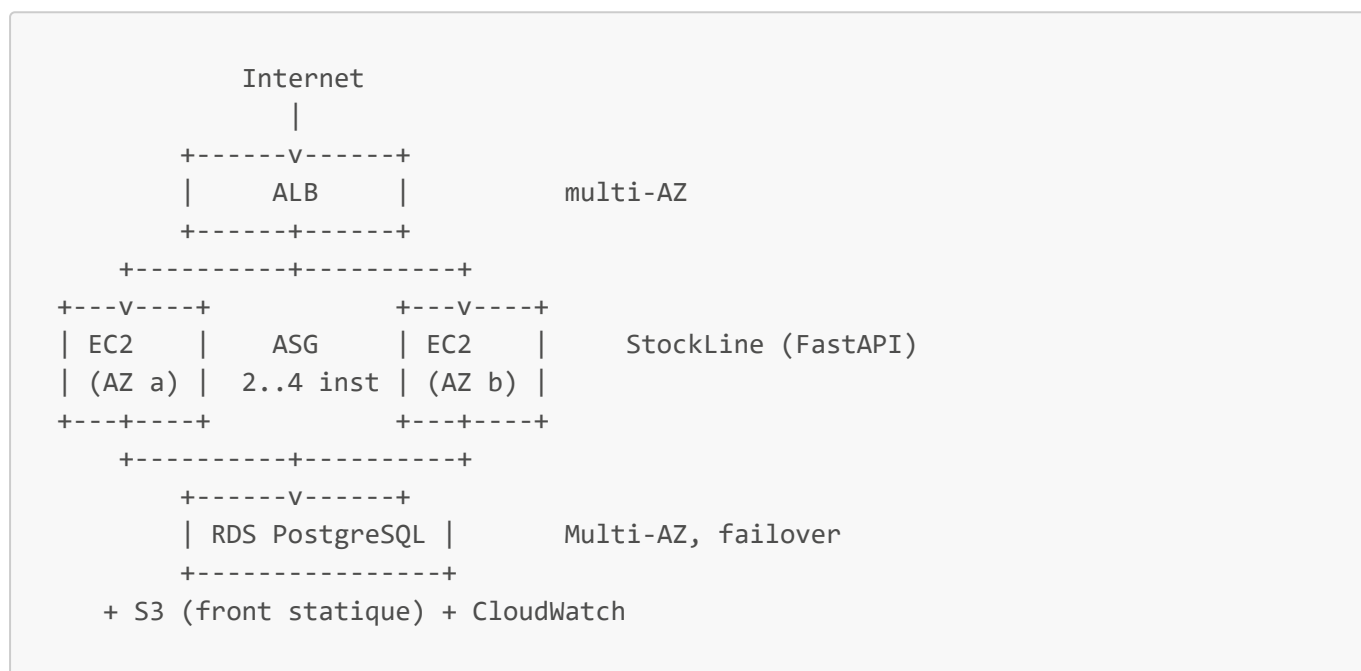
Restitution : chaque binôme présente UNE architecture (5 min), le groupe challenge les chiffres.

Fichier : [exercices/08-cl-aws2/exercice-6-1-facture-negoce.md](#) Objectif du DAF : -40 %. Un bon audit trouve **plus**.

5. Architectures de référence — et migration

Trois façons de déployer StockLine, côte à côte

Référence 1 — le 3-tiers (votre TP2)

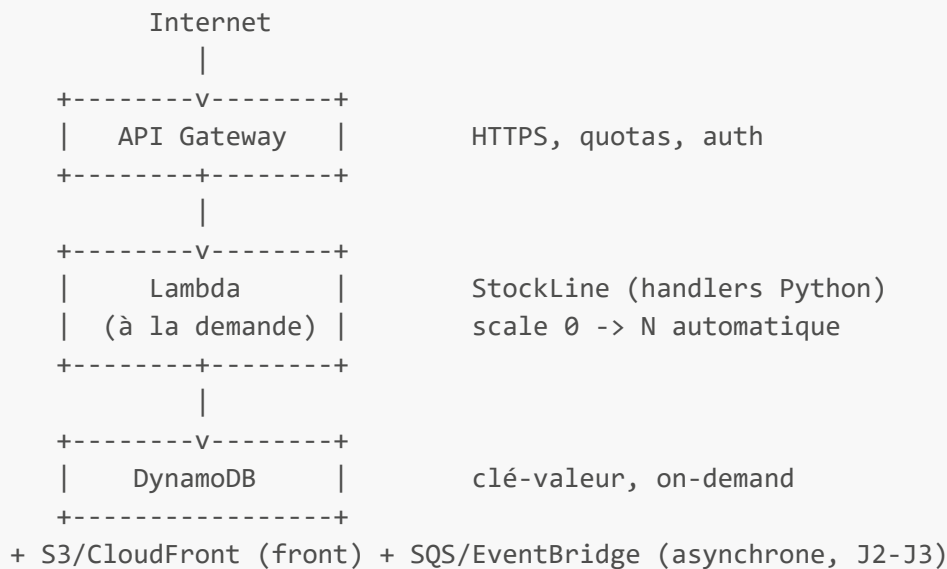


Le 3-tiers : forces, faiblesses, coûts

Forces	maîtrise totale (OS, tuning), pattern universel, aucune limite de durée/protocole, compétences très répandues
Faiblesses	vous administrez tout (patching, AMI, capacité), coût plancher élevé (ça tourne même à 0 requête), scaling en minutes
Coût type	~150-500 / <i>mois même petit</i> : <i>ALB</i> (19) + 2 instances + RDS Multi-AZ + NAT — le coût fixe domine
Idéal pour	charge soutenue et prévisible, applications existantes migrées telles quelles, besoins spécifiques d'OS

C'est l'architecture « lift and shift » par excellence : celle de Negoce+ (architecture A), celle de votre TP2 — et celle qu'on optimise avec Savings Plans + right-sizing.

Référence 2 — le serverless (votre TP3, dans 2 semaines)

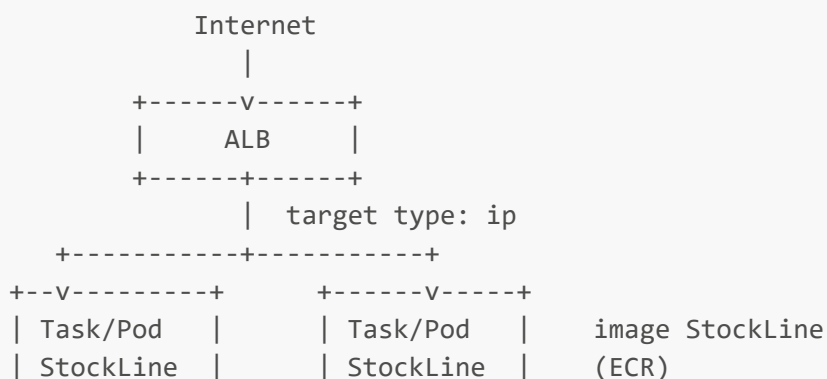


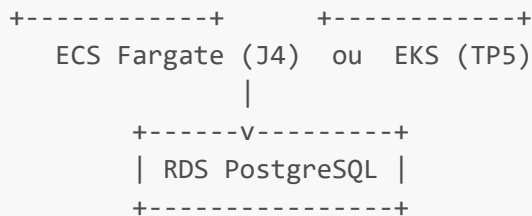
Le serverless : forces, faiblesses, coûts

Forces	coût ≈ 0 au repos , scaling automatique et instantané, zéro serveur à administrer, haute disponibilité incluse
Faiblesses	15 min max par exécution, cold starts, adhérence AWS forte, modèle de données à repenser (DynamoDB ≠ SQL), débogage distribué
Coût type	à l'usage pur : 1 M de requêtes API + Lambda + DynamoDB ≈ quelques \$/mois ; MAIS peut dépasser le 3-tiers à très fort trafic soutenu
Idéal pour	trafic irrégulier ou faible, APIs événementielles, automatisations, démarrages de projets

La règle de coût à retenir : **serverless gagne quand le trafic est creux ou en pics ; le fixe (3-tiers/Fargate) gagne quand le trafic est massif et constant**. Le point de bascule se **calcule** — comme à l'exercice 4-2.

Référence 3 — les conteneurs (votre TP5, au bloc 14)





Les conteneurs : forces, faiblesses, coûts

Forces	portabilité (poste → cloud → autre cloud), déploiements rapides et réversibles, bonne densité, standard du marché (Kubernetes)
Faiblesses	courbe d'apprentissage (bloc 13 : 7 jours !), coût plancher non nul (tâches allumées), la base de données reste à part (RDS)
Coût type	intermédiaire : 2 tâches Fargate 0,5 vCPU + ALB + RDS ≈ 100-250 /mois; <i>EK Sajoute 73</i> /mois de plan de contrôle
Idéal pour	équipes produits, microservices, portabilité exigée, charge continue avec pics modérés

Synthèse des 3 : il n'existe **pas de meilleure architecture** — il existe un meilleur choix **par charge, par équipe, par budget**. Savoir le justifier chiffres à l'appui : c'est exactement ce qu'on attendra de vous en entretien (et au TP3 : comparatif argumenté).

Migrer vers le cloud : les 6R en bref

Dernier outil de la boîte : quand une entreprise migre son existant, chaque application passe par une décision parmi **6 stratégies** :

R	Stratégie	En clair	Exemple cursus
Rehost	« lift and shift »	la VM telle quelle sur EC2	StockLine TP1 → TP2
Replatform	« lift and reshape »	petits gains sans refonte	base locale → RDS
Repurchase	racheter en SaaS	abandonner l'app pour un service	paie maison → SaaS RH
Refactor	ré-architecturer	repenser cloud-native	StockLine → serverless (TP3)
Retire	décommissionner	l'app ne sert plus : on éteint	10-20 % d'un parc, souvent
Retain	garder sur place	contrainte légale/technique	vieux progiciel métier

À l'examen Cloud Practitioner, on vous décrira une situation et vous choisirez le bon R. Dans la vraie vie : un **portefeuille** d'applications = un mélange des 6.



Exercice 6-2 — Le portefeuille de Negoce+ et les 6R

Individuel, 25 minutes.

La DSI de Negoce+ liste 6 applications au-delà de StockLine (messagerie interne, vieux progiciel de comptabilité, site vitrine, outil de BI, serveur de fichiers, application RH).

Pour chacune : choisissez **la stratégie R adaptée**, en une phrase de justification — et pour deux d'entre elles, le **service AWS cible**.

Fichier : [exercices/08-cl-aws2/exercice-6-2-migration-6r.md](#)

Correction collective rapide avant le quiz — plusieurs réponses sont défendables, l'important est **l'argument**.

Récap avant le grand quiz

- **Well-Architected** : 6 piliers — vous les pratiquez déjà ; sachez les **nommer** et illustrer chacun d'un exemple du cursus.
- **Tarification** : compute / stockage / réseau. EC2 : on-demand, Savings Plans, reserved, **spot**. S3 : classes + **lifecycle**. Le piège : **les Go sortants** et la NAT GW.
- **Outils** : Cost Explorer (voir), Budgets (alerter), **tags** (attribuer), Trusted Advisor / Compute Optimizer (recommander).
- **Audit** : la checklist des 10 gaspillages — éteindre, redimensionner, nettoyer, s'engager.
- **3 architectures de référence** : 3-tiers (fixe, maîtrise), serverless (usage, 0 au repos), conteneurs (portabilité) — le choix se **justifie et se chiffre**.
- **6R** : Rehost, Repatform, Repurchase, Refactor, Retire, Retain.

Et maintenant : 20 questions, format Cloud Practitioner, sur **tout AWS1 + AWS2**.



Grand quiz de synthèse AWS

20 questions — conditions d'examen : seul, sans notes, ~25 minutes

Format CLF-C02 : une seule bonne réponse par question. Correction commentée juste après.

Grand quiz — questions 1 et 2

Question 1. Dans le modèle de responsabilité partagée, qui est responsable du chiffrement des données stockées dans S3 ?

A. AWS, toujours B. Le client : la sécurité **DANS** le cloud est sa responsabilité C. Personne, S3 n'est pas chiffrable D. L'hébergeur du datacenter

Question 2. Une politique IAM refuse (**Deny**) une action qu'une autre politique autorise (**Allow**). Résultat ?

A. L'action est autorisée, **Allow** gagne B. L'action est refusée : un **Deny** explicite l'emporte toujours C. Cela dépend de l'ordre des politiques D. AWS demande une confirmation

Grand quiz — questions 3 et 4

Question 3. Quelle est la portée d'une région AWS par rapport aux zones de disponibilité (AZ) ?

- A. Une AZ contient plusieurs régions
- B. Une région contient plusieurs AZ, datacenters isolés reliés en réseau rapide
- C. Région et AZ sont synonymes
- D. Les AZ ne servent qu'au stockage S3

Question 4. Votre application sur EC2 doit lire un bucket S3. La bonne pratique est :

- A. Copier des clés d'accès IAM dans un fichier sur l'instance
 - B. Rendre le bucket public en lecture
 - C. Attacher un rôle IAM à l'instance via un instance profile
 - D. Utiliser le compte root pour générer une clé permanente
-

Grand quiz — questions 5 et 6

Question 5. Quelle différence essentielle entre un security group et une NACL ?

- A. Le SG est stateful (les réponses passent automatiquement), la NACL est stateless
- B. La NACL protège les instances, le SG protège le sous-réseau
- C. Le SG peut contenir des règles Deny explicites
- D. Aucune, ce sont deux noms du même objet

Question 6. Une instance dans un sous-réseau privé doit télécharger des mises à jour depuis Internet. Il lui faut :

- A. Une Internet Gateway attachée directement à l'instance
 - B. Une route vers une NAT Gateway placée dans un sous-réseau public
 - C. Une adresse IPv4 publique élastique
 - D. Un enregistrement Route 53
-

Grand quiz — questions 7 et 8

Question 7. Quel service fournit une base de données **relationnelle managée** avec bascule automatique en cas de panne d'AZ ?

- A. DynamoDB en mode global
- B. RDS en déploiement Multi-AZ
- C. S3 avec versioning
- D. ElastiCache

Question 8. À quoi sert un Auto Scaling Group ?

- A. À répartir le trafic HTTP entre des instances
 - B. À maintenir un nombre d'instances et l'ajuster automatiquement selon la charge
 - C. À sauvegarder les instances chaque nuit
 - D. À réserver de la capacité pour 3 ans
-

Grand quiz — questions 9 et 10

Question 9. Quelle classe S3 est la plus adaptée à des archives légales consultées au plus une fois tous les 5 ans, au moindre coût ?

- A. S3 Standard
- B. S3 Standard-IA
- C. S3 Glacier Deep Archive
- D. S3 Intelligent-Tiering

Question 10. Quel service enregistre **qui a fait quel appel d'API** sur le compte AWS (audit) ?

- A. CloudWatch
 - B. CloudTrail
 - C. Cost Explorer
 - D. Trusted Advisor
-

Grand quiz — questions 11 et 12

Question 11. Une fonction Lambda a besoin d'écrire dans une table DynamoDB. Où lui donne-t-on ce droit ?

A. Dans le code Python, avec des clés d'accès B. Dans son rôle d'exécution IAM C. Dans le security group de la fonction D. Dans la politique du bucket S3

Question 12. Quel service permet de **découpler** deux applications en stockant les messages jusqu'à leur traitement ?

A. SNS B. SQS C. CloudFront D. Route 53

Grand quiz — questions 13 et 14

Question 13. Différence fondamentale entre SNS et SQS :

A. SNS pousse un message vers plusieurs abonnés (pub/sub) ; SQS est une file qu'un consommateur vient lire
B. SQS pousse, SNS stocke C. SNS ne fonctionne qu'avec les e-mails D. SQS est déprécié au profit de SNS

Question 14. Pour orchestrer un enchaînement de plusieurs Lambda avec gestion d'erreurs et reprises, le service adapté est :

A. EventBridge B. Step Functions C. CodePipeline D. API Gateway

Grand quiz — questions 15 et 16

Question 15. Dans ECS, l'objet qui garantit que N tâches restent en vie et les rattache au load balancer est :

A. La task definition B. Le cluster C. Le service D. Le repository ECR

Question 16. Principal avantage de Fargate sur le launch type EC2 :

A. C'est toujours moins cher B. Aucune instance à administrer : on paie la tâche à la seconde C. Il permet d'installer des agents sur l'OS hôte D. Il fonctionne sans VPC

Grand quiz — questions 17 et 18

Question 17. Dans CloudFormation, avant d'appliquer une mise à jour sur une stack de production, la bonne pratique est :

A. Supprimer la stack et la recréer B. Créer un changeset et vérifier la colonne Replacement C. Modifier les ressources en console puis mettre à jour le template D. Désactiver le rollback

Question 18. Un traitement batch reprennable tourne 6 h par nuit et tolère les interruptions. Le modèle d'achat EC2 le plus économique est :

A. On-Demand B. Reserved 3 ans C. Spot D. Dedicated Host

Grand quiz — questions 19 et 20

Question 19. Quel poste de facturation est le plus souvent sous-estimé dans une architecture AWS ?

A. Les données entrantes, facturées au Go B. Les données sortantes vers Internet et le trafic NAT/inter-AZ C. Le nombre d'utilisateurs IAM D. Les tags de facturation

Question 20. Negoce+ migre son vieux progiciel de comptabilité, non modifiable, sur une VM EC2 identique à son serveur actuel. Quelle stratégie des 6R ?

A. Refactor B. Repurchase C. Rehost D. Retire

Rendez vos copies ! Correction commentée dans 10 minutes — comptez vos points : $\geq 14/20$ = niveau Cloud Practitioner en bonne voie.

Clôture du bloc CL-AWS2

En 6 jours, vous avez ajouté à votre boîte à outils AWS :

- le **serverless** : Lambda, API Gateway, DynamoDB, SQS/SNS, Step Functions, EventBridge ;
- les **conteneurs managés** : ECR, ECS, Fargate — en attendant Docker/Kubernetes au bloc 13 ;
- l'**infrastructure as code** : CloudFormation — en attendant Terraform au bloc 11 ;
- la **maîtrise des coûts** : le réflexe qui différencie un junior recruté d'un junior écarté.

Teardown de fin de bloc, maintenant et ensemble (checklist dans le guide) : stacks CloudFormation, tâches ECS, ALB, tables DynamoDB, fonctions Lambda de la semaine. `aws resourcegroupstaggingapi get-resources` pour la contre-vérification finale.

À lundi — direction Azure !

Bloc 09 : Azure fondamentaux (6 jours)

Vous parlez maintenant couramment AWS. Bonne nouvelle : **les concepts sont transposables** — abonnements ↔ comptes, groupes de ressources, VNets ↔ VPC, NSG ↔ security groups, App Service, Azure Monitor...

Le multi-cloud n'est pas un luxe : c'est ce que demandent les DSI françaises. Et au TP3, vous déploierez StockLine **sur les deux**.

Reposez-vous — et vérifiez une dernière fois votre facture. 💰